Assignment-8

—----------

Name: Soham Chavan
PRN: 124B1F022
Class: SY IT A
Date: 15/04/2026

—----------

Aim: No SQL: Aggregation and Indexing: Design and Develop Queries using aggregation and indexing with suitable example using Mongo DB.

Introduction

Theory: MongoDB Aggregation is a pipeline-based framework for processing and transforming documents. Data passes through a sequence of stages — each stage filters, reshapes, groups, or sorts the documents. Indexing in MongoDB creates a special data structure that speeds up queries by allowing MongoDB to find documents without scanning every document in a collection.

Part A — Aggregation Pipeline Stages

The aggregation pipeline passes documents through stages one by one. Common stages are:

Stage	Syntax	What It Does
$match	{ $match: { field: value } }	Filter documents — like SQL WHERE
$group	{ $group: { _id: '$field', ... } }	Group docs and compute totals, averages, counts
$project	{ $project: { field:1, _id:0 } }	Choose which fields to include or exclude
$sort	{ $sort: { field: 1 or -1 } }	Sort output (1=ascending, -1=descending)
$limit	{ $limit: N }	Keep only the first N documents
$skip	{ $skip: N }	Skip the first N documents
$count	{ $count: 'total' }	Count how many documents pass through
$unwind	{ $unwind: '$arrayField' }	Flatten an array into separate documents
$lookup	{ $lookup: { from, localField, ... }}	Join another collection (like SQL JOIN)
$addFields	{ $addFields: { newField: expr } }	Add a computed field to each document

Part B — Index Types

Indexes are created with createIndex(). MongoDB supports several index types:

Index Type	Example	Use Case
Single Field	createIndex({email:1})	Speed up queries on one field
Compound	createIndex({status:1,price:-1})	Speed up queries on multiple fields
Unique	createIndex({email:1},{unique:true})	Prevent duplicate values
Text	createIndex({name:'text'})	Full-text search on string fields

Index Type	Example	Use Case
TTL (Expiry)	createIndex({createdAt:1},{expireAfterSeconds:N})	Auto-delete documents after time
Multikey	createIndex({tags:1})	Index array field — one entry per array element

DATABASE 1: airlinedb — Airline Management System

Sample Data

▶ Code

```
use airlinedb

db.airlines.insertMany([
  { _id:1, name:'IndiGo',    iata:'6E', country:'India' },
  { _id:2, name:'Air India', iata:'AI', country:'India' },
  { _id:3, name:'SpiceJet',  iata:'SG', country:'India' }
])

db.flights.insertMany([
  { _id:1, airline_id:1, flight_no:'6E-101', origin:'Mumbai',
    destination:'Delhi',     seats:180, available:45, status:'Scheduled' },
  { _id:2, airline_id:2, flight_no:'AI-202', origin:'Delhi',
    destination:'Bangalore', seats:200, available:0,  status:'Scheduled' },
  { _id:3, airline_id:1, flight_no:'6E-305', origin:'Mumbai',
    destination:'Chennai',   seats:180, available:90, status:'Scheduled' },
  { _id:4, airline_id:3, flight_no:'SG-303', origin:'Chennai',
    destination:'Kolkata',   seats:150, available:20, status:'Delayed'   }
])

db.passengers.insertMany([
  { _id:1, name:'Arjun Sharma', email:'arjun@mail.com', city:'Mumbai' },
  { _id:2, name:'Priya Verma',  email:'priya@mail.com', city:'Delhi'  },
  { _id:3, name:'Rahul Singh',  email:'rahul@mail.com', city:'Mumbai' },
  { _id:4, name:'Sneha Patel',  email:'sneha@mail.com', city:'Pune'   }
])

db.bookings.insertMany([
  { _id:1, passenger_id:1, flight_id:1, class:'Economy',  status:'Confirmed' },
  { _id:2, passenger_id:2, flight_id:2, class:'Business', status:'Confirmed' },
  { _id:3, passenger_id:3, flight_id:1, class:'Economy',  status:'Confirmed' },
  { _id:4, passenger_id:1, flight_id:3, class:'Business', status:'Confirmed' },
  { _id:5, passenger_id:4, flight_id:4, class:'Economy',  status:'Cancelled' },
  { _id:6, passenger_id:2, flight_id:3, class:'Economy',  status:'Confirmed' }
])

db.payments.insertMany([
  { _id:1, booking_id:1, amount:4500,  status:'Success' },
  { _id:2, booking_id:2, amount:12000, status:'Success' },
  { _id:3, booking_id:3, amount:4500,  status:'Success' },
  { _id:4, booking_id:4, amount:9000,  status:'Success' },
  { _id:5, booking_id:5, amount:3800,  status:'Refunded'},
  { _id:6, booking_id:6, amount:4200,  status:'Success' }
])
```

▶ Output

```
{ acknowledged: true, insertedCount: 3 }   // airlines
{ acknowledged: true, insertedCount: 4 }   // flights
{ acknowledged: true, insertedCount: 4 }   // passengers
{ acknowledged: true, insertedCount: 6 }   // bookings
{ acknowledged: true, insertedCount: 6 }   // payments
```

AGGREGATION — airlinedb

A1 — $count: Count Total Bookings

Theory: $count is the simplest aggregation stage. It counts how many documents pass through the pipeline and stores the result in a named field. Here we first use $match to filter only Confirmed bookings before counting.

▶ **Code**

```
// Count all bookings
db.bookings.aggregate([
  { $count: 'total_bookings' }
])

// Count only Confirmed bookings
db.bookings.aggregate([
  { $match:  { status: 'Confirmed' } },
  { $count:  'confirmed_bookings'   }
])
```

▶ **Output**

```
// All bookings:
{ total_bookings: 6 }

// Confirmed only:
{ confirmed_bookings: 5 }
```

A2 — $group + $sum: Bookings per Flight

Theory: $group groups documents by a key (_id field) and applies accumulator expressions like $sum to count or total values within each group. This is equivalent to SQL's GROUP BY with COUNT(*).

▶ **Code**

```
db.bookings.aggregate([
  {
    $group: {
      _id:              '$flight_id',
      total_bookings:  { $sum: 1 },
      confirmed_count: { $sum: { $cond: [{ $eq:['$status','Confirmed'] }, 1, 0] } }
    }
  },
  { $sort: { total_bookings: -1 } }
])
```

▶ **Output**

```
{ _id: 1, total_bookings: 2, confirmed_count: 2 }
{ _id: 3, total_bookings: 2, confirmed_count: 2 }
{ _id: 2, total_bookings: 1, confirmed_count: 1 }
{ _id: 4, total_bookings: 1, confirmed_count: 0 }
```

A3 — $group + $sum: Total Revenue from Payments

Theory: By grouping payments on a constant (_id: null), we aggregate across all documents. $sum adds up all amount values; $avg computes the mean. This gives a single summary row — equivalent to SQL's SELECT SUM(amount), AVG(amount).

▶ **Code**

```
// Total revenue from successful payments
db.payments.aggregate([
  { $match: { status: 'Success' } },
  {
    $group: {
      _id:           null,
      total_revenue: { $sum: '$amount' },
      avg_ticket:    { $avg: '$amount' },
      max_payment:   { $max: '$amount' },
      min_payment:   { $min: '$amount' }
    }
  },
  {
    $project: {
      _id:           0,
```

```
      total_revenue: 1,
      avg_ticket:    { $round: ['$avg_ticket', 2] },
      max_payment:   1,
      min_payment:   1
    }
  }
])
```

▶ **Output**

```
{
  total_revenue: 34200,
  avg_ticket:    6840.00,
  max_payment:   12000,
  min_payment:   4200
}
```

A4 — $group + $avg: Average Revenue per Booking Class

Theory: Grouping by the class field inside bookings, then joining with payments using $lookup, lets us calculate the average ticket price per travel class — useful for pricing analysis.

▶ **Code**

```
db.bookings.aggregate([
  { $match: { status: 'Confirmed' } },
  {
    $lookup: {
      from:         'payments',
      localField:   '_id',
      foreignField: 'booking_id',
      as:           'pay_info'
    }
  },
  { $unwind: '$pay_info' },
  {
    $group: {
      _id:         '$class',
      avg_price:   { $avg:  '$pay_info.amount' },
      total_revenue:{ $sum: '$pay_info.amount' },
      count:       { $sum: 1 }
    }
  },
  {
    $project: {
      class:         '$_id',
      avg_price:     { $round: ['$avg_price', 2] },
      total_revenue: 1,
      count:         1,
      _id:           0
    }
  }
])
```

▶ **Output**

```
{ class:'Business', avg_price:10500.00, total_revenue:21000, count:2 }
{ class:'Economy',  avg_price: 4400.00, total_revenue:13200, count:3 }
```

A5 — $sort + $limit: Top Paying Passenger

Theory: Combining $lookup to fetch passenger names, $group to sum their payments, $sort to order by spend, and $limit to keep only the top result is the aggregation equivalent of SQL's ORDER BY ... LIMIT 1.

▶ **Code**

```
db.bookings.aggregate([
  { $match: { status: 'Confirmed' } },
  {
```

```
    $lookup: {
      from: 'payments', localField: '_id',
      foreignField: 'booking_id', as: 'pay'
    }
  },
  { $unwind: '$pay' },
  {
    $group: {
      _id:       '$passenger_id',
      total_paid:{ $sum: '$pay.amount' }
    }
  },
  { $sort:  { total_paid: -1 } },
  { $limit: 3 },
  {
    $lookup: {
      from: 'passengers', localField: '_id',
      foreignField: '_id', as: 'pax'
    }
  },
  { $unwind: '$pax' },
  { $project: { name:'$pax.name', total_paid:1, _id:0 } }
])
```

▶ Output

```
{ name:'Arjun Sharma', total_paid: 13500 }
{ name:'Priya Verma',  total_paid: 16200 }
{ name:'Rahul Singh',  total_paid:  4500 }
```

A6 — $addFields + $project: Compute Load Factor

Theory: $addFields adds a new computed field to every document without removing existing ones. Here we compute the load_factor (percentage of seats booked) for each flight using a simple arithmetic expression.

▶ Code

```
db.flights.aggregate([
  {
    $addFields: {
      booked_seats: { $subtract: ['$seats', '$available'] },
      load_factor: {
        $round: [
          { $multiply: [
              { $divide: [{ $subtract:['$seats','$available'] }, '$seats'] },
              100
          ]},
          2
        ]
      }
    }
  },
  {
    $project: {
      flight_no:    1,
      origin:       1,
      destination:  1,
      booked_seats: 1,
      load_factor:  1,
      _id:          0
    }
  },
  { $sort: { load_factor: -1 } }
])
```

▶ Output

```
{ flight_no:'AI-202', origin:'Delhi',   destination:'Bangalore', booked_seats:200,
load_factor:100   }
```

```
{ flight_no:'SG-303', origin:'Chennai', destination:'Kolkata',   booked_seats:130,
load_factor:86.67 }
{ flight_no:'6E-101', origin:'Mumbai',  destination:'Delhi',     booked_seats:135,
load_factor:75    }
{ flight_no:'6E-305', origin:'Mumbai',  destination:'Chennai',   booked_seats:90,
load_factor:50    }
```

INDEXING — airlinedb

Theory: Without indexes, MongoDB scans every document in a collection (called a COLLSCAN) for each query. An index builds a sorted structure on a field so MongoDB can jump directly to matching documents (IXSCAN). Use explain('executionStats') to see whether an index was used.

Index A1 — Single Field Index on email (passengers)

Theory: A single field index on email speeds up login lookups. We add unique:true to prevent two passengers sharing the same email address — enforcing a uniqueness constraint at the database level.

▶ **Code**

```
// Create unique index on email
db.passengers.createIndex({ email: 1 }, { unique: true })

// List all indexes on passengers
db.passengers.getIndexes()

// Query that USES the index
db.passengers.find({ email: 'arjun@mail.com' })

// Check if index was used
db.passengers.find({ email: 'arjun@mail.com' }).explain('executionStats')
```

▶ **Output**

```
// createIndex result:
{ numIndexesBefore: 1, numIndexesAfter: 2, createdCollectionAutomatically: false, ok: 1 }

// getIndexes result:
[ { key:{ _id:1 }, name:'_id_' },
  { key:{ email:1 }, name:'email_1', unique:true } ]

// explain stageName (shows index was used):
winningPlan.stage: 'IXSCAN'
indexName: 'email_1'
totalDocsExamined: 1      // only 1 doc scanned — very fast!
```

Index A2 — Single Field Index on flight_id (bookings)

Theory: Bookings are frequently queried by flight_id to list all passengers on a flight. A single index on flight_id makes this lookup instant regardless of how large the bookings collection grows.

▶ **Code**

```
db.bookings.createIndex({ flight_id: 1 })

// Query that uses this index
db.bookings.find({ flight_id: 1 })
           .explain('executionStats')
```

▶ **Output**

```
{ numIndexesBefore:1, numIndexesAfter:2, ok:1 }

// explain output:
winningPlan.stage: 'IXSCAN'
indexName: 'flight_id_1'
totalDocsExamined: 2     // only 2 bookings on flight_id=1
totalKeysExamined: 2
```

Index A3 — Compound Index on (status, class) for Bookings

Theory: A compound index covers queries that filter on two fields together. The order matters: put the most selective field first. This index speeds up queries like 'find all Confirmed Economy bookings'

▶ Code

```
// Compound index: status first, then class
db.bookings.createIndex({ status: 1, class: 1 })

// Query covered by compound index
db.bookings.find({ status: 'Confirmed', class: 'Economy' })
           .explain('executionStats')

// Also works for prefix queries (status only)
db.bookings.find({ status: 'Confirmed' })
```

▶ Output

```
{ numIndexesBefore:2, numIndexesAfter:3, ok:1 }

// explain (status+class query):
winningPlan.stage: 'IXSCAN'
indexName: 'status_1_class_1'
totalDocsExamined: 3

// Results:
{ _id:1, passenger_id:1, flight_id:1, class:'Economy', status:'Confirmed' }
{ _id:3, passenger_id:3, flight_id:1, class:'Economy', status:'Confirmed' }
{ _id:6, passenger_id:2, flight_id:3, class:'Economy', status:'Confirmed' }
```

Index A4 — dropIndex and listIndexes

Theory: You can drop an index that is no longer needed using dropIndex(). This frees storage and removes the overhead of maintaining the index on writes. getIndexes() lists all current indexes on a collection.

▶ Code

```
// List all indexes before drop
db.bookings.getIndexes()

// Drop compound index by name
db.bookings.dropIndex('status_1_class_1')

// List indexes after drop
db.bookings.getIndexes()
```

▶ Output

```
// Before drop — 3 indexes:
[ { name:'_id_' }, { name:'flight_id_1' }, { name:'status_1_class_1' } ]

// dropIndex result:
{ nIndexesWas: 3, ok: 1 }

// After drop — 2 indexes:
[ { name:'_id_' }, { name:'flight_id_1' } ]
```

DATABASE 2: shopsphere — E-Commerce Platform

Sample Data

▶ Code

```
use shopsphere

db.categories.insertMany([
  { _id:1, name:'Electronics' },
  { _id:2, name:'Clothing'    },
  { _id:3, name:'Books'       }
])

db.products.insertMany([
  { _id:1, name:'Smartphone X',  cat_id:1, price:25000, stock:50,  rating:4.5 },
  { _id:2, name:'Laptop Pro',    cat_id:1, price:75000, stock:20,  rating:4.7 },
  { _id:3, name:'Cotton T-Shirt',cat_id:2, price:599,   stock:200, rating:4.2 },
  { _id:4, name:'DBMS Textbook', cat_id:3, price:850,   stock:100, rating:4.8 },
  { _id:5, name:'Headphones',    cat_id:1, price:3500,  stock:80,  rating:4.3 }
])

db.users.insertMany([
  { _id:1, name:'Alice Roy',   email:'alice@mail.com', city:'Mumbai'    },
  { _id:2, name:'Bob Das',     email:'bob@mail.com',   city:'Delhi'     },
  { _id:3, name:'Carol Nair',  email:'carol@mail.com', city:'Bangalore' }
])

db.orders.insertMany([
  { _id:1, user_id:1, status:'Delivered', total:25599,
    items:[{product_id:1,qty:1,price:25000},{product_id:3,qty:1,price:599}] },
  { _id:2, user_id:2, status:'Delivered', total:75000,
    items:[{product_id:2,qty:1,price:75000}] },
  { _id:3, user_id:3, status:'Pending',   total:1198,
    items:[{product_id:3,qty:2,price:599}] },
  { _id:4, user_id:1, status:'Delivered', total:3500,
    items:[{product_id:5,qty:1,price:3500}] },
  { _id:5, user_id:2, status:'Cancelled', total:850,
    items:[{product_id:4,qty:1,price:850}] }
])

db.reviews.insertMany([
  { _id:1, product_id:1, user_id:2, rating:5, comment:'Excellent!' },
  { _id:2, product_id:2, user_id:1, rating:4, comment:'Great.'     },
  { _id:3, product_id:3, user_id:3, rating:4, comment:'Good.'      },
  { _id:4, product_id:4, user_id:1, rating:5, comment:'Best book.' },
  { _id:5, product_id:1, user_id:3, rating:4, comment:'Nice phone.'}
])
```

▶ Output

```
{ acknowledged: true, insertedCount: 3 }   // categories
{ acknowledged: true, insertedCount: 5 }   // products
{ acknowledged: true, insertedCount: 3 }   // users
{ acknowledged: true, insertedCount: 5 }   // orders
{ acknowledged: true, insertedCount: 5 }   // reviews
```

AGGREGATION — shopsphere

S1 — $group + $sum: Total Sales per Category

Theory: $unwind flattens the items array inside each order so each item becomes its own document. After unwinding, we can group by product category, sum revenue, and count units sold — a classic sales report.

▶ Code

```
db.orders.aggregate([
  { $match: { status: { $ne: 'Cancelled' } } },
  { $unwind: '$items' },
  {
    $lookup: {
      from: 'products', localField: 'items.product_id',
      foreignField: '_id', as: 'product'
    }
  },
  { $unwind: '$product' },
  {
    $group: {
      _id:        '$product.cat_id',
      total_sales:{ $sum: { $multiply: ['$items.qty', '$items.price'] } },
      units_sold: { $sum: '$items.qty' }
    }
  },
  {
    $lookup: {
      from: 'categories', localField: '_id',
      foreignField: '_id', as: 'cat'
    }
  },
  { $unwind: '$cat' },
  { $project: { category:'$cat.name', total_sales:1, units_sold:1, _id:0 } },
  { $sort: { total_sales: -1 } }
])
```

▶ Output

```
{ category:'Electronics', total_sales:103500, units_sold:3 }
{ category:'Clothing',    total_sales:  1797, units_sold:3 }
```

S2 — $group + $avg: Average Product Rating per Category

Theory: By grouping the products collection by cat_id and computing $avg on the rating field, we get the mean rating for each category. $round then formats the result to two decimal places.

▶ Code

```
db.products.aggregate([
  {
    $group: {
      _id:         '$cat_id',
      avg_rating:  { $avg:  '$rating' },
      total_stock: { $sum:  '$stock'  },
      num_products:{ $sum:  1         },
      max_price:   { $max:  '$price'  }
    }
  },
  {
    $lookup: {
      from:'categories', localField:'_id',
      foreignField:'_id', as:'cat'
    }
  },
  { $unwind: '$cat' },
  {
    $project: {
      category:    '$cat.name',
      avg_rating:  { $round:['$avg_rating', 2] },
      total_stock: 1,
```

```
      num_products:1,
      max_price:   1,
      _id:         0
    }
  }
])
```

▶ **Output**

```
{ category:'Books',       avg_rating:4.8, total_stock:100, num_products:1, max_price:850
}
{ category:'Electronics', avg_rating:4.5, total_stock:150, num_products:3, max_price:75000
}
{ category:'Clothing',    avg_rating:4.2, total_stock:200, num_products:1, max_price:599
}
```

S3 — $group: Customer Order Summary (Lifetime Value)

Theory: This query groups orders by user_id, sums total spend, counts orders, and then joins with users to get names. The result is a customer report showing lifetime value — very common in e-commerce analytics.

▶ **Code**

```
db.orders.aggregate([
  { $match: { status: { $ne: 'Cancelled' } } },
  {
    $group: {
      _id:         '$user_id',
      total_spent: { $sum: '$total' },
      num_orders:  { $sum: 1 }
    }
  },
  { $sort: { total_spent: -1 } },
  {
    $lookup: {
      from:'users', localField:'_id',
      foreignField:'_id', as:'user'
    }
  },
  { $unwind: '$user' },
  {
    $project: {
      customer:    '$user.name',
      total_spent: 1,
      num_orders:  1,
      _id:         0
    }
  }
])
```

▶ **Output**

```
{ customer:'Bob Das',   total_spent:75000, num_orders:1 }
{ customer:'Alice Roy', total_spent:29099, num_orders:2 }
{ customer:'Carol Nair',total_spent: 1198, num_orders:1 }
```

S4 — $group: Average Review Rating per Product

Theory: Grouping reviews by product_id and computing $avg gives each product's average rating from actual reviews. Comparing this with the stored product rating field helps detect stale data.

▶ **Code**

```
db.reviews.aggregate([
  {
    $group: {
      _id:          '$product_id',
      avg_rating:   { $avg: '$rating' },
      review_count: { $sum: 1 }
```

```
    }
  },
  {
    $lookup: {
      from:'products', localField:'_id',
      foreignField:'_id', as:'prod'
    }
  },
  { $unwind: '$prod' },
  {
    $project: {
      product:      '$prod.name',
      avg_rating:   { $round:['$avg_rating', 1] },
      review_count: 1,
      _id:          0
    }
  },
  { $sort: { avg_rating: -1 } }
])
```

▶ **Output**

```
{ product:'DBMS Textbook',  avg_rating:5.0, review_count:1 }
{ product:'Smartphone X',   avg_rating:4.5, review_count:2 }
{ product:'Laptop Pro',     avg_rating:4.0, review_count:1 }
{ product:'Cotton T-Shirt', avg_rating:4.0, review_count:1 }
```

S5 — $match + $sort + $limit: Top 3 Most Expensive Products

Theory: A simple three-stage pipeline: $match filters in-stock products, $sort orders by price descending, and $limit keeps only the top 3. This is one of the most common aggregation patterns.

▶ **Code**

```
db.products.aggregate([
  { $match: { stock: { $gt: 0 } } },
  { $sort:  { price: -1 } },
  { $limit: 3 },
  {
    $project: {
      name:  1,
      price: 1,
      stock: 1,
      _id:   0
    }
  }
])
```

▶ **Output**

```
{ name:'Laptop Pro',   price:75000, stock:20 }
{ name:'Smartphone X', price:25000, stock:50 }
{ name:'Headphones',   price:3500,  stock:80 }
```

S6 — $addFields: Add Discount Price Field

Theory: $addFields computes a new field for every document without a separate update operation. Here we add a discounted_price field calculated as 10% off the original price — useful for promotional catalogue views.

▶ **Code**

```
db.products.aggregate([
  {
    $addFields: {
      discounted_price: {
        $round: [{ $multiply: ['$price', 0.90] }, 2]
      },
      discount_amount: {
        $round: [{ $multiply: ['$price', 0.10] }, 2]
```

```
      }
    }
  },
  {
    $project: {
      name:             1,
      original_price:   '$price',
      discounted_price: 1,
      discount_amount:  1,
      _id:              0
    }
  }
])
```

▶ Output

```
{ name:'Smartphone X',   original_price:25000, discounted_price:22500,
discount_amount:2500 }
{ name:'Laptop Pro',     original_price:75000, discounted_price:67500,
discount_amount:7500 }
{ name:'Cotton T-Shirt', original_price:599,   discounted_price:539.1,
discount_amount:59.9 }
{ name:'DBMS Textbook',  original_price:850,   discounted_price:765,   discount_amount:85
}
{ name:'Headphones',     original_price:3500,  discounted_price:3150,  discount_amount:350
}
```

INDEXING — shopsphere

Theory: For an e-commerce platform, the most important indexes are on email (for user login), product name and category (for search and browse), order status (for fulfilment dashboards), and reviews.product_id (for rating calculations).

Index S1 — Unique Index on users.email

Theory: Every e-commerce platform needs fast login by email and must prevent two accounts sharing the same email. A unique single-field index on email solves both requirements with one command.

▶ **Code**

```
db.users.createIndex({ email: 1 }, { unique: true })

// Verify
db.users.getIndexes()

// Fast login query
db.users.find({ email: 'alice@mail.com' })
```

▶ **Output**

```
{ numIndexesBefore:1, numIndexesAfter:2, ok:1 }

// Indexes:
[ { key:{ _id:1 }, name:'_id_' },
  { key:{ email:1 }, name:'email_1', unique:true } ]

// Query result:
{ _id:1, name:'Alice Roy', email:'alice@mail.com', city:'Mumbai' }
```

Index S2 — Single Field Index on products.cat_id

Theory: Browse-by-category is the most common product query. An index on cat_id makes fetching all Electronics, Clothing, or Books products instant — regardless of how many products exist in total.

▶ **Code**

```
db.products.createIndex({ cat_id: 1 })

// Query all Electronics
db.products.find({ cat_id: 1 }, { name:1, price:1, _id:0 })
           .explain('executionStats')
```

▶ **Output**

```
{ numIndexesBefore:1, numIndexesAfter:2, ok:1 }

// explain:
winningPlan.stage: 'IXSCAN'
indexName: 'cat_id_1'
totalDocsExamined: 3   // only 3 Electronics products scanned

// Query results:
{ name:'Smartphone X', price:25000 }
{ name:'Laptop Pro',   price:75000 }
{ name:'Headphones',   price:3500  }
```

Index S3 — Compound Index on orders (user_id + status)

Theory: Order history pages query by user_id and often filter by status. A compound index on (user_id, status) covers both patterns. The prefix rule means it also speeds up queries on user_id alone.

▶ **Code**

```
db.orders.createIndex({ user_id: 1, status: 1 })
```

```
// Query: Alice's Delivered orders
db.orders.find({ user_id:1, status:'Delivered' },
               { total:1, status:1, _id:0 })

// Also works: Alice's orders (any status)
db.orders.find({ user_id: 1 }, { total:1, status:1, _id:0 })
```

▶ **Output**

```
{ numIndexesBefore:1, numIndexesAfter:2, ok:1 }

// user_id=1, status=Delivered:
{ total:25599, status:'Delivered' }
{ total:3500,  status:'Delivered' }

// user_id=1 only:
{ total:25599, status:'Delivered' }
{ total:3500,  status:'Delivered' }
{ total:850,   status:'Cancelled' }
```

Index S4 — Text Index on products.name (Full-Text Search)

Theory: A text index lets users search for products by keyword using $text and $search. MongoDB tokenises the field, removes stop words, and builds a word-level index. Only one text index is allowed per collection.

▶ **Code**

```
db.products.createIndex({ name: 'text' })

// Search for products containing 'pro'
db.products.find(
  { $text: { $search: 'pro' } },
  { name:1, price:1, _id:0 }
)

// Search for 'shirt' OR 'book'
db.products.find(
  { $text: { $search: 'shirt book' } },
  { name:1, price:1, _id:0 }
)
```

▶ **Output**

```
{ numIndexesBefore:2, numIndexesAfter:3, ok:1 }

// Search 'pro':
{ name:'Laptop Pro', price:75000 }

// Search 'shirt book':
{ name:'DBMS Textbook',  price:850 }
{ name:'Cotton T-Shirt', price:599 }
```

Index S5 — Multikey Index on order items (embedded array)

Theory: When a field contains an array, MongoDB automatically creates a multikey index — one index entry per array element. This allows efficient queries on the items.product_id field inside the embedded order items array.

▶ **Code**

```
// Index on the product_id inside the embedded items array
db.orders.createIndex({ 'items.product_id': 1 })

// Find all orders containing Smartphone X (product_id=1)
db.orders.find(
  { 'items.product_id': 1 },
  { user_id:1, status:1, total:1, _id:0 }
```

```
)

// Check indexes
db.orders.getIndexes()
```

▶ Output

```
{ numIndexesBefore:2, numIndexesAfter:3, ok:1 }

// Orders containing Smartphone X:
{ user_id:1, status:'Delivered', total:25599 }

// getIndexes:
[ { name:'_id_' },
  { name:'user_id_1_status_1' },
  { name:'items.product_id_1', multikey:true } ]
```

Summary of All Queries

All aggregation and indexing queries across both databases:

#	Ref	Type	Database	Description
1	**A1**	Aggregation	airlinedb	$count — total bookings and confirmed bookings
2	**A2**	Aggregation	airlinedb	$group + $sum — bookings per flight
3	**A3**	Aggregation	airlinedb	$group + $sum/$avg/$max/$min — payment summary
4	**A4**	Aggregation	airlinedb	$group + $avg — revenue per booking class ($lookup)
5	**A5**	Aggregation	airlinedb	$sort + $limit — top paying passengers
6	**A6**	Aggregation	airlinedb	$addFields — compute load factor for each flight
7	**A-I1**	Indexing	airlinedb	Unique index on passengers.email
8	**A-I2**	Indexing	airlinedb	Single index on bookings.flight_id
9	**A-I3**	Indexing	airlinedb	Compound index on bookings (status, class)
10	**A-I4**	Indexing	airlinedb	dropIndex and getIndexes
11	**S1**	Aggregation	shopsphere	$unwind + $group — total sales per category
12	**S2**	Aggregation	shopsphere	$group + $avg — avg rating per category
13	**S3**	Aggregation	shopsphere	$group — customer lifetime value report
14	**S4**	Aggregation	shopsphere	$group — average review rating per product
15	**S5**	Aggregation	shopsphere	$match + $sort + $limit — top 3 products
16	**S6**	Aggregation	shopsphere	$addFields — compute discounted price (10% off)
17	**S-I1**	Indexing	shopsphere	Unique index on users.email
18	**S-I2**	Indexing	shopsphere	Single index on products.cat_id
19	**S-I3**	Indexing	shopsphere	Compound index on orders (user_id, status)
20	**S-I4**	Indexing	shopsphere	Text index on products.name for keyword search
21	**S-I5**	Indexing	shopsphere	Multikey index on embedded items.product_id

Conclusion

Through this assignment, I managed to see how the MongoDB's aggregation pipeline and indexing make data handling easier. Aggregation is the process of transforming and manipulating data into meaningful information through a series of processes such as matching, groupings, sorting, and joining collections.

Some of the operations that were utilized in order to manipulate the data include match, which filters documents; group, which calculates the average and total values; sort and limit, which sorts documents; and lookup, which combines two collections. Also, there was the use of project, add fields, and unwind.

Besides aggregation, various types of indexes have been learned such as single field index, compound index, text index, and multikey index. These types of indexes enable efficient querying since they do not require complete scanning of entire collections.

The primary lesson to draw from this assignment is the creation of indexes for often used fields and utilization of match operation in order to achieve better results during aggregation operations. Nonetheless, too many indexes need to be avoided since they will negatively impact the operations.